NATIONAL ECONOMICS UNIVERSITY
FACULTY OF DATA SCIENCE AND ARTIFICIAL INTELLIGENCE

GROUP ASSIGNMENT REPORT

Midterm

Web Design and Programming (CNTT1164)

Rùa Toán Basic: A Mathematics Practice Website

A beginner-friendly implementation with HTML, CSS, JavaScript, FastAPI, and PostgreSQL

Group: Group 09
Class: EP16.CNTT1164(126)_01
Instructor: M.Sc. Le Duy Khanh
Semester: Semester 1, 2026–2027
GitHub: https://github.com/thunopro/ruatoan_basic
Demo: https://ruatoanbasic.vercel.app

Team Members

No.	Full name	Student ID	Role
1	Nguyen Khac Thu	11248480	Team lead, Full-stack Architecture, Backend, Frontend, Database, Deployment
2	Le Quang	11247343	Team member, UI/UX Review, Responsive Testing
3	Huy Minh	11248482	Team member, QA Testing, Edge-case Bug Triage

Hanoi, November 2026

Declaration

We declare that this report is the result of our group's own work for the course Web Design and Programming. All sources (literature, source code, images, and AI tools) are cited explicitly. We take full responsibility for the submitted content.

Task Allocation and Contributions

Member	Work done	Contribution (%)
Nguyen Khac Thu (Lead)	Architecture, FastAPI Backend, Frontend Implementation, PostgreSQL, Deployment	100%
Le Quang	UI/UX Review, Cross-browser & Responsive Testing, Report review	0%
Huy Minh	Quality Assurance, Search/Filter Edge-case Bug Triage, Report review	0%
	Total	**100%**

Table of Contents

1 Introduction . . . 4
1.1 Project background . . . 4
1.2 Problem statement . . . 4
1.3 Objectives and scope . . . 4
1.4 Intended users . . . 4
2 Analysis and Design . . . 4
2.1 Requirements . . . 5
2.1.1 Functional requirements . . . 5
2.1.2 Non-functional requirements . . . 5
2.2 Interface design . . . 6
2.3 System architecture . . . 6
2.4 Database design . . . 6
3 Implementation . . . 7
3.1 Project structure . . . 7
3.2 Browser to backend request . . . 7
3.3 Reading the content files . . . 8
3.4 Account and session flow . . . 8
3.5 Photo submission flow . . . 8
3.6 Bookmarks and page behaviour . . . 8
3.7 Running locally . . . 9
3.8 Deployment design . . . 9
4 Results and Conclusion . . . 9
4.1 Implemented result . . . 9
4.2 Verification status and limitations . . . 10
4.3 Conclusion . . . 10
4.4 Future improvements . . . 10
References . . . 12
A Appendix: API and Data Flow . . . 12
A.1 Main API routes . . . 12
A.2 Simple request example . . . 12
A.3 Environment configuration . . . 12

List of Figures

1 Basic application architecture and the two data stores. 6
2 Simple relationship between a user and uploaded images. 7

List of Tables

1 Three PostgreSQL tables created for the basic project. 11
2 Main API routes in the basic project. 12

1. Introduction

1.1. Project background

Rùa Toán is a website for students who want to practise mathematics, browse exam collections, review theory, and find learning materials. The existing production project uses a modern TypeScript and React-based stack. Rùa Toán Basic is a separate learning version of the same idea. It uses plain HTML pages, CSS stylesheets, browser JavaScript, and a small FastAPI backend so that a junior developer can follow the code one file at a time.

The basic project reuses prepared learning content from the current Rùa Toán repository. At the time the data snapshot was copied, it contained 3,381 unique collections with 33,929 unique problem records, 90 lessons, 216 books or magazines, and 89 documents. The problem records are grouped inside their collections. These figures describe the copied data files; they do not mean each item has been manually reviewed in this project.

1.2. Problem statement

A student needs one place to find practice questions, open full collections, review lessons, and access learning material. The site should work on a phone or computer and should let a student create an account, sign in, sign out, and keep uploaded photographs of written work. The code should be understandable to someone who has first studied basic HTML, CSS, JavaScript, HTTP requests, and simple Python APIs.

1.3. Objectives and scope

The project has four practical objectives:

1. Present a clear home page and navigation between learning pages.
2. Read the prepared mathematics catalogue and provide search, grade selection, pagination, and collection detail pages.
3. Demonstrate account registration, password-based login, logout, and account-owned image storage.
4. Explain the data flow and project structure with short code modules and simple commands.

The catalogue is read-only in this version. Students do not create or edit the canonical question bank. A question can be bookmarked in the current browser using `localStorage`; this browser bookmark is not a cloud-synced account feature. Uploaded photos and account records are stored in PostgreSQL. The basic version uses email and password rather than Google OAuth.

1.4. Intended users

The main users are lower-secondary students preparing for mathematics tests, teachers or tutors who want to browse collections, and junior developers learning how a browser page communicates with a Python API and a database.

2. Analysis and Design

2.1. Requirements

The requirements below describe the implemented basic version, not every advanced feature in the original production website.

2.1.1. Functional requirements

FR1. Show the home page, navigation, deterministic daily problem (`/api/daily`), and counts for collections, problems, lessons, books, and documents.

FR2. Search individual questions by text (with Vietnamese accent-insensitive matching), grade, and canonical topic tabs; list exercise collections by category (*HSG*, *Vào 10*, *Tạp chí*, *Chuyên đề*, *Olympic*), search by title or province, and move through paginated results.

FR3. Open a collection and read its questions in their original order inside a full-width printable exam sheet, including formatted Markdown, mathematical notation, per-question homework photo uploads, and source attribution when available.

FR4. List lessons with topic tabs, search lesson titles or summaries (accent-insensitive), and expand lessons inline to read full Markdown content and practice examples.

FR5. List books, magazines, and PDF documents with topic filtering, search, and external/download links.

FR6. Register with a name, email, and password; sign in and sign out; change password or request a password reset token; show the signed-in name in the shared navigation.

FR7. Compress selected homework photos in the browser (grayscale WebP up to 1000 px), upload up to six JPG, PNG, or WebP files per request (optionally linked to a specific problem ID), and view (with Lightbox preview) or delete photos belonging to the signed-in account.

FR8. Save problem bookmarks in browser storage (`localStorage`) and show/manage the saved list on the home page on that same browser.

2.1.2. Non-functional requirements

- **Easy to learn:** HTML describes page content, CSS controls appearance, JavaScript handles browser events, and FastAPI defines API routes.
- **Usable on different screens:** the stylesheet reduces grid columns and stacks the home hero on narrow screens. This is a source-level design goal; a formal cross-device audit has not been recorded.
- **Maintainable:** each page has its own HTML and JavaScript file, and database, authentication, and content code are kept in separate Python modules.
- **Safe password handling:** the application stores a salted PBKDF2 hash rather than the original password and uses a signed HttpOnly session cookie.
- **Private photo access:** photo endpoints check the signed-in user ID before reading or deleting an image.
- **Bounded upload:** the API accepts up to six images per request, supports JPEG, PNG, and WebP, and limits each image to 1.5 MB in application code.
- **Data durability:** PostgreSQL is used for accounts and photos because files written by a serverless function are not a durable database. The deployed application therefore needs a configured PostgreSQL service and environment variables.

- **Content correctness:** the JSON data includes source names and links when provided. The application does not claim that every source transcription or link has been independently checked.

2.2. Interface design

The interface ports the signature "Chibi Ninja Turtle" retro arcade theme from the original Rùa Toán application via `original.css` and `style.css`: a deep forest-green pixel-grid background, warm parchment scroll cards (`.scroll`), custom SVG Ninja Turtle mascots representing the four math clans (*Số học*, *Đại số*, *Hình học*, *Tổ hợp*), and tactile 3D arcade buttons. A shared navigation bar appears on the home page and all learning pages, highlighting the active route and updating the account area based on `/api/me`.

The home page introduces the four math clans, features the deterministic daily problem with inline homework photo upload, displays catalogue totals, and lists saved browser bookmarks. The practice page provides interactive topic tabs, a search field, a grade selector, and paginated problem cards. The exam page provides category tabs, search/grade filters, and a full-width printable exam view (`.de`) that keeps questions in source order. The lesson and library pages provide topic tabs and accent-insensitive search fields. The login and registration pages provide tab-switched authentication forms and an inline password-reset panel. The submission page contains a client-side WebP image compressor/uploader, a gallery with Lightbox preview, and a password-change card.

Problem and lesson text is sanitized and formatted by a safe client-side Markdown helper (`renderMarkdown`) before MathJax renders dollar-delimited expressions. If the external math-rendering script is unavailable, the plain mathematical notation remains visible.

2.3. System architecture

Figure 1 shows the main request path. The browser downloads static HTML, CSS, JavaScript, and JSON files. Browser JavaScript calls the FastAPI routes under `/api`. FastAPI reads the exercise catalogue from JSON files packaged with the project. For registration, sessions, and uploaded photos it connects to PostgreSQL. The database tables belong to the separate `ruatoan_basic` schema.

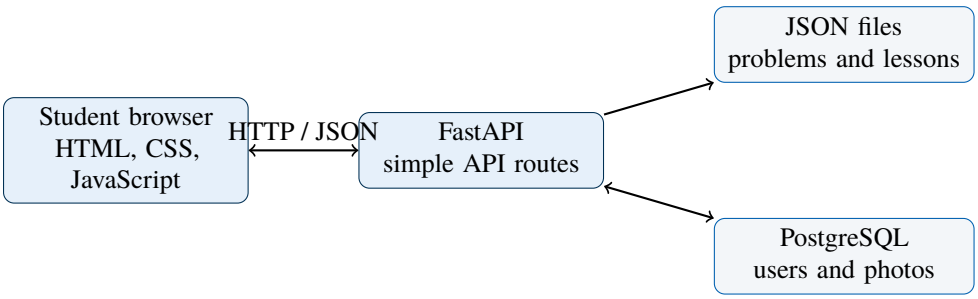


Figure 1. Basic application architecture and the two data stores.

2.4. Database design

The project intentionally does not put the full question catalogue into PostgreSQL. Exercise and lesson content is already prepared as JSON and is read-only in this version. PostgreSQL stores data that changes while students use the application: account records and submitted images.

The image is stored as PostgreSQL `BYTEA`, a binary field. This keeps the first version easy to understand and means photo access can be restricted using the account relationship. For a large public service, object storage

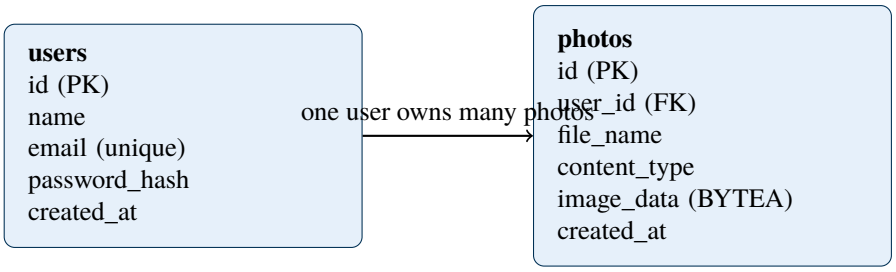


Figure 2. Simple relationship between a user and uploaded images.

is often more suitable for many large image files; that scaling change is outside this beginner project.

3. Implementation

3.1. Project structure

The folders follow the basic learning path: first HTML pages, then CSS and JavaScript, then a small Python backend.

Listing 1. Rùa Toán Basic folder layout.

```
ruatoan_basic/
|-- main.py                 # FastAPI entry point
|-- backend/
|   |-- main.py             # API paths and page responses
|   |-- database.py         # PostgreSQL connection and 3 tables
|   |-- auth.py             # PBKDF2 hash, signed cookie, reset token hash
|   `-- content.py          # read JSON files and fold Vietnamese accents
|-- public/
|   |-- index.html          # home page
|   |-- pages/              # practice, exams, lessons, library, auth, submissions
|   |-- css/original.css    # ported design tokens and arcade components
|   |-- css/style.css       # responsive layout, lightbox, and page styles
|   |-- js/                 # browser actions (common.js + one file per page)
|   `-- images/             # website images
|-- data/                   # exercise, lesson, book, and document JSON
|-- scripts/                # smoke_test.py and local PostgreSQL helper scripts
|-- requirements.txt        # Python packages
|-- .env.example            # names of required environment values
|-- render.yaml             # Render Blueprint deployment configuration
`-- vercel.json             # Vercel routing and function settings
```

3.2. Browser to backend request

A page starts as HTML. The browser downloads its linked CSS and JavaScript files. JavaScript listens for events such as a form submission or button click. It then calls a FastAPI URL with `fetch()`, reads a JSON response, and updates the page. For example, the practice page sends the search text and selected grade to `GET /api/collections`; FastAPI filters the list and returns one page of results.

This separation has a small number of concepts: HTML elements, CSS selectors, JavaScript event handlers, an HTTP request, a Python function, and a JSON response. No frontend framework is required.

3.3. Reading the content files

`backend/content.py` loads the prepared JSON files from `data/` (or `public/data/`). It combines collection files, keeps the collection copy with more questions when an ID repeats, and removes a question ID already shown in an earlier collection. Lessons are combined from their JSON files. Books and documents are read from their own files. A simple in-memory cache avoids parsing every JSON file for every API request in one running process.

The catalogue is read-only. The API returns up to 50 collections or problems per list request. Search across collections, problems, lessons, books, and documents folds Vietnamese accents (`fold_text`) so queries with or without diacritics match reliably. Problem search also maps canonical topic slugs (`so-hoc`, `dai-so`, `hinh-hoc`, `to-hop`) to their underlying dataset aliases.

3.4. Account and session flow

The registration page sends a name, email, and password to `POST /api/register`. The backend requires a password of at least eight characters, creates a random salt, and uses Python's PBKDF2-HMAC-SHA256 to produce the value saved in `password_hash`. It inserts the account into PostgreSQL, then issues a signed cookie. Login reads the account by email and compares a new password hash with the stored hash using a constant-time comparison. Logout expires the cookie. The browser asks `GET /api/me` to decide whether to show the signed-in name or a login link. Signed-in users can also update their password via `POST /api/change-password`, or reset a forgotten password via `POST /api/request-reset` and `POST /api/reset-password` (which stores a SHA-256 hash of a single-use token expiring in 30 minutes; raw tokens are only returned in responses when `PASSWORD_RESET_DEBUG=true` for local testing).

The cookie contains an account ID and expiry time with an HMAC signature. The browser cannot read the cookie through JavaScript because the cookie is marked HttpOnly. The cookie is secure on HTTPS and uses SameSite=Lax.

3.5. Photo submission flow

Before uploading, the browser-side helper `compressImage()` in `public/js/common.js` draws each selected image onto an HTML5 canvas, scales its longest side down to at most 1000 pixels, converts it to grayscale for high-contrast handwriting legibility, and encodes it as a WebP blob (`quality = 0.82`). The submission page (as well as the inline upload buttons on the daily problem and exam detail pages) sends the resulting files to `POST /api/photos` using `FormData`, optionally tagging the upload with `problem_id`. The API requires a signed-in user, accepts up to six JPEG, PNG, or WebP files, and limits each file to 1.5 MB. The file bytes are inserted into the PostgreSQL `BYTEA` column. Listing, viewing, and deletion queries include the current user ID, so one account cannot request another account's image by guessing a photo ID.

3.6. Bookmarks and page behaviour

Problem bookmarks are saved in browser `localStorage` and displayed in a dedicated "Bài đã lưu" section on the home page as well as toggleable bookmark buttons on problem cards. The practice page provides interactive topic tabs, a grade selector, accent-insensitive text search, and pagination. The exam collection page filters by exam category (*HSG*, *Vào 10*, *Tạp chí*, *Chuyên đề*, *Olympic*), search query, and grade, and opens individual exams in a full-width printable view (`.de`) with source order preserved. Lessons can be filtered by topic, searched, and expanded inline with full Markdown and practice examples. Books and documents can be filtered by topic and text and opened or downloaded via their recorded external URLs.

3.7. Running locally

The README gives these beginner commands after Python and PostgreSQL settings have been prepared:

Listing 2. Install and run on a development computer.

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
# Add DATABASE_URL and SESSION_SECRET to .env.local
set -a
source .env.local
set +a
uvicorn main:app --reload
```

Then open `http://127.0.0.1:8000`. The first registration creates the schema and the two tables. The database URL and session secret are environment configuration and must not be included in source control.

3.8. Deployment design

The project includes a root `main.py` that exports the FastAPI application for Vercel. The deployed service requires `DATABASE_URL` and `SESSION_SECRET` to be configured in the Vercel project settings. The JSON content is packaged with the app; account and photo persistence is delegated to PostgreSQL. Vercel documents support for FastAPI and notes that function filesystems are read-only apart from temporary scratch space, so a local file is not a persistent account database [1, 2].

4. Results and Conclusion

4.1. Implemented result

Rùa Toán Basic provides a complete, beginner-friendly implementation with these connected parts:

- A plain HTML home page and separate HTML pages for practice, exam collections, lessons, library, sign-in, registration, and submissions.
- Ported retro-arcade design tokens (`original.css`) paired with responsive page styles (`style.css`), custom Chibi Ninja Turtle SVG mascots, safe Markdown + MathJax rendering, Lightbox photo preview, and browser-side grayscale WebP image compression (`common.js`).
- FastAPI endpoints for catalogue data, deterministic daily problems, account actions, password change/reset, current-session lookup, and account-owned photo actions (including per-problem photo attachments).
- JSON content for questions, lessons, books, and documents; three PostgreSQL tables (`users`, `photos`, `password_resets`) for user accounts, submitted homework photos, and password reset tokens.
- Deployment configurations for both Render (`render.yaml`) and Vercel (`vercel.json`), local PostgreSQL scripts, and an automated smoke test suite (`scripts/smoke_test.py`).

The catalogue scale is useful for demonstrating that a read-only set of thousands of collections can remain as prepared JSON while the smaller changing user records are stored in PostgreSQL. It does not establish that JSON is faster than PostgreSQL for every workload; a performance comparison would require a defined dataset, equivalent queries, and measured tests.

4.2. Verification status and limitations

This report documents source-level behaviour. A production smoke check confirmed the home page, CSS/JavaScript files, content APIs, and reported catalogue counts. A temporary account was registered, signed in, signed out, and removed. A small image was uploaded to PostgreSQL, retrieved byte-for-byte, and deleted. These checks cover basic request paths only; no load test or performance benchmark is claimed.

The current basic project is intended to cover the same main learning activities, but it is not a feature-for-feature replacement of the current Rùa Toán website. It uses email/password rather than Google sign-in; it has fewer filters; bookmarks are browser-local; image data is stored directly in PostgreSQL; and it omits the original site's automated content crawling, advanced topic relationships, and some presentation details. These limits keep the code shorter and easier for a beginner to trace.

4.3. Conclusion

The project demonstrates how a familiar website can be built with foundational web technologies. HTML defines each page, CSS styles it, JavaScript sends requests and handles interaction, FastAPI provides server routes, JSON supplies read-only learning content, and PostgreSQL stores account and image records that must persist. Keeping these jobs in separate beginner-sized files makes the data flow visible and provides a starting point for later improvements.

4.4. Future improvements

Possible next steps are adding topic and year filters, syncing bookmarks to an account, resizing images in the browser, storing large photos in object storage, adding password reset and email verification, and adding a small performance measurement section. Each change can be introduced as one page or one API route at a time.

Table	Important fields	Purpose
`ruatoan_basic.users`	`id`, `name`, `email`, `password_hash`, `created_at`	One row per registered account; email is unique.
`ruatoan_basic.photos`	`id`, `user_id`, `problem_id`, `file_name`, `content_type`, `image_data`, `created_at`	Stores image bytes, optional `problem` reference, and metadata. `user_id` links each image to its owner.
`ruatoan_basic.password_resets`	`token_hash`, `user_id`, `expires_at`, `created_at`	Stores SHA-256 hashed password reset tokens with a 30-minute expiration.

Table 1. Three PostgreSQL tables created for the basic project.

References

[1] Vercel. Fastapi on vercel, 2026. Accessed: 2026-11-01.

[2] Vercel. Vercel functions runtimes, 2026. Accessed: 2026-11-01.

A. Appendix: API and Data Flow

A.1. Main API routes

Method	Path	Purpose
GET	`/api/stats`, `/api/daily`	Return catalogue counts and the deterministic daily problem.
GET	`/api/collections`	Search (accent-insensitive) and filter collection records by kind, grade, and page.
GET	`/api/collections/{id}`	Return one collection with ordered problems and source metadata.
GET	`/api/problems`	Search individual questions by text, grade, canonical topic, and page.
GET	`/api/lessons`	Return or search lesson records (with examples and Markdown body).
GET	`/api/books`, `/api/docs`	Return or search books, magazines, and PDF documents.
POST	`/api/register`, `/api/login`	Create account or start a signed HttpOnly cookie session.
POST	`/api/logout`; GET `/api/me`	End the session or read the signed-in account.
POST	`/api/change-password`	Change password for the signed-in account.
POST	`/api/request-reset`, `/api/reset-password`	Generate a 30-minute reset token and set a new password.
GET/POST	`/api/photos`	List photos (optionally filtered by `problem`) or upload compressed photos.
GET/DELETE	`/api/photos/{id}`	View (`nosniff`) or delete an account-owned image.

Table 2. Main API routes in the basic project.

A.2. Simple request example

When a student searches for a collection, the browser reads the search text and grade selection. It makes a URL such as `/api/collections?search=Ha+Noi&grade=9&page=1`. FastAPI reads these query parameters, selects matching JSON records, and returns JSON. The browser then creates collection cards from the returned records. In the photo flow, the browser sends file bytes instead of JSON text; the API checks the session and stores the bytes in PostgreSQL.

A.3. Environment configuration

The application needs two private environment values: `DATABASE_URL`, the connection string from the PostgreSQL provider, and `SESSION_SECRET`, a long random signing value. A sample names-only file is provided as `.env.example`. The real local `.env.local` file is ignored by Git. Neither private value should be copied into this report, screenshots, or public source control.